



PERTEMUAN 13

Search dan Sort

Memahami algoritma pencarian dan pengurutan data — dua fondasi penting dalam ilmu komputer yang digunakan hampir di setiap aplikasi modern.



Pengenalan: Search & Sort

Pencarian dan pengurutan adalah dua operasi paling mendasar dalam pemrosesan data. Memahami keduanya membantu kita memilih algoritma yang tepat sesuai kebutuhan.

Pencarian (Search)

Mencari apakah suatu nilai (kunci pencarian) ada dalam kumpulan data, dan jika ada, menemukan lokasinya. Dua algoritma populer: **Linear Search** (sederhana) dan **Binary Search** (lebih cepat).

Pengurutan (Sort)

Menempatkan data ke dalam urutan tertentu — **menaik (ascending)** atau **menurun (descending)** — berdasarkan satu atau beberapa kunci. Contoh: nama diurutkan abjad, rekening diurutkan berdasarkan nomor akun.



Linear Search

Linear Search adalah algoritma pencarian paling sederhana — ia memeriksa elemen satu per satu dari awal hingga akhir array.

1 Periksa Satu per Satu

Algoritma memeriksa setiap elemen array secara berurutan, mulai dari indeks pertama hingga terakhir.

2 Kunci Ditemukan

Jika kunci pencarian cocok dengan suatu elemen, algoritma segera mengembalikan **indeks** elemen tersebut dan berhenti.

3 Kunci Tidak Ditemukan

Jika seluruh array sudah diperiksa dan tidak ada yang cocok, algoritma memberitahu bahwa kunci pencarian **tidak ada** dalam array.

 Bayangkan mencari nama di daftar hadir — kamu membaca satu per satu dari atas ke bawah. Itulah cara kerja Linear Search!

Big O Notation

Semua algoritma pencarian memiliki tujuan yang sama — menemukan elemen yang cocok. Tapi bagaimana cara kita membandingkan **seberapa efisien** suatu algoritma? Jawabannya adalah **Big O Notation**.

Apa itu Big O?

Big O Notation adalah cara matematis untuk menggambarkan **seberapa keras** sebuah algoritma harus bekerja. Ini menunjukkan bagaimana waktu eksekusi bertambah seiring bertambahnya jumlah data (n).

Mengapa Penting?

Big O membantu kita memilih algoritma terbaik. Sebuah algoritma yang lambat bisa sangat merugikan saat data berjumlah jutaan elemen, sementara algoritma efisien tetap cepat.

Fokus pada Pertumbuhan

Big O mengabaikan konstanta kecil dan hanya memperhatikan **suku dominan** — bagian yang paling mempengaruhi performa saat n membesar.



O(1) — Waktu Konstan

Artinya Apa?

Algoritma dengan kompleksitas $O(1)$ selalu membutuhkan jumlah operasi yang **sama**, tidak peduli seberapa besar ukuran datanya. Ini adalah efisiensi terbaik yang mungkin!

$O(1)$ dibaca: "*orde satu*" atau "*waktu konstan*"

Contoh Nyata

Misalkan kita ingin memeriksa apakah elemen pertama array sama dengan elemen kedua:

- Array 10 elemen → butuh **1 perbandingan**
- Array 1.000 elemen → butuh **1 perbandingan**
- Array 1.000.000 elemen → tetap **1 perbandingan!**

Bahkan jika kita membandingkan elemen pertama dengan 3 elemen berikutnya (3 perbandingan), algoritma tetap $O(1)$ karena jumlahnya **tidak bergantung pada ukuran n**.



$O(n)$ — Waktu Linear

Algoritma $O(n)$ memiliki jumlah operasi yang bertumbuh **sebanding** dengan jumlah elemen. Jika data dua kali lipat, waktu eksekusi pun kira-kira dua kali lipat.

Contoh Kasus

Memeriksa apakah elemen pertama array sama dengan *salah satu* elemen lainnya memerlukan paling banyak $n - 1$ perbandingan.

Pertumbuhan Linier

Array 10 elemen → hingga **9 perbandingan**

Array 1.000 elemen → hingga **999 perbandingan**

Saat n besar, nilai "-1" tidak berarti → disederhanakan menjadi $O(n)$.

Prinsip Big O

Big O menyederhanakan dengan mengabaikan suku kecil. Ekspresi $n-1$ cukup diwakili oleh n , karena "1" tidak relevan saat n sangat besar.

$O(n^2)$ — Waktu Kuadratik

Algoritma $O(n^2)$ jauh lebih lambat. Jika data dua kali lipat, waktu eksekusi menjadi **empat kali lipat**. Ini terjadi ketika ada loop bersarang (loop di dalam loop).

Contoh: Mencari Duplikat

Bayangkan kita ingin memeriksa apakah ada elemen yang duplikat dalam array:

- Elemen ke-1 dibandingkan dengan semua elemen lain $\rightarrow n-1$ perbandingan
- Elemen ke-2 dibandingkan dengan sisanya $\rightarrow n-2$ perbandingan
- Dan seterusnya hingga elemen terakhir...

Total: $(n-1) + (n-2) + \dots + 1 = n^2/2 - n/2$

Mengapa Disebut $O(n^2)$?

Saat n membesar, suku $n^2/2$ mendominasi. Big O mengabaikan konstanta ($\div 2$) dan suku kecil ($-n/2$), sehingga hasilnya adalah $O(n^2)$.

Algoritma ini disebut memiliki waktu **kuadratik** — efisiensinya menurun drastis untuk data besar!

Big O dari Linear Search



Kompleksitas: $O(n)$

Linear Search berjalan dalam waktu $O(n)$ — artinya semakin banyak elemen, semakin banyak perbandingan yang harus dilakukan.



Skalabilitas

Jika ukuran array **dua kali lipat**, jumlah perbandingan yang diperlukan juga **dua kali lipat**. Performanya konsisten tapi tidak efisien untuk data besar.



Kasus Terburuk

Kasus terburuk terjadi ketika kunci pencarian berada di **akhir array** atau tidak ada sama sekali — seluruh array harus diperiksa satu per satu.

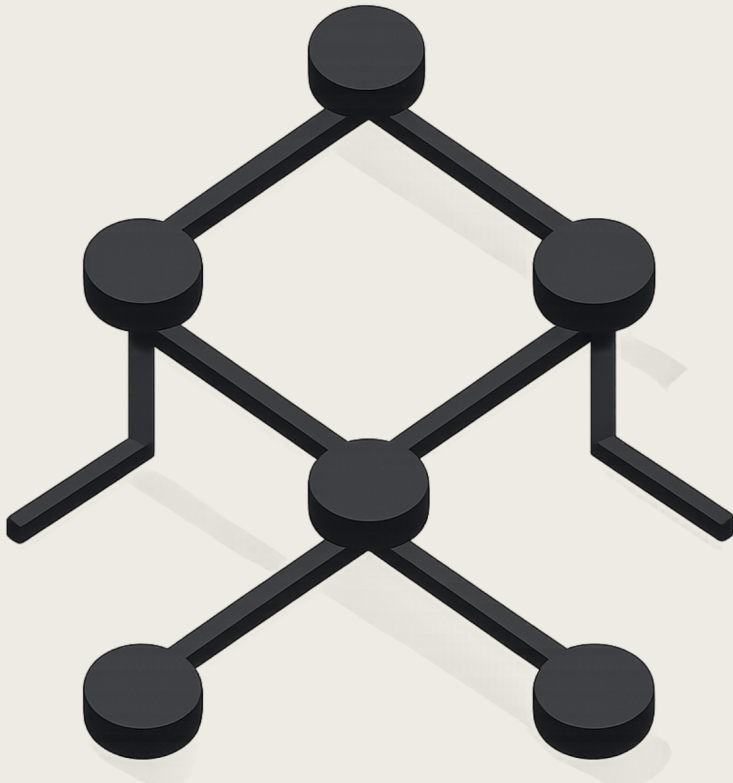


Kapan Linear Search Oke?

Bisa sangat cepat jika elemen yang dicari kebetulan ada di **awal array**. Cocok untuk data kecil atau data yang tidak terurut.

Binary Search

Binary Search jauh lebih efisien dari Linear Search, namun dengan satu syarat penting: **array harus sudah diurutkan terlebih dahulu**.



1

Cek Elemen Tengah

Algoritma memeriksa elemen di posisi **tengah** array. Jika cocok dengan kunci pencarian, selesai!

2

Kurang dari Tengah?

Jika kunci lebih kecil, berarti kunci ada di **bagian kiri**. Bagian kanan langsung diabaikan seluruhnya.

3

Lebih dari Tengah?

Jika kunci lebih besar, berarti kunci ada di **bagian kanan**. Bagian kiri langsung diabaikan seluruhnya.

4

Ulangi!

Proses ini diulang pada subarray yang tersisa hingga kunci ditemukan atau array habis.



Bayangkan mencari kata di kamus — kamu langsung buka halaman tengah, lalu cari ke kiri atau kanan. Itu Binary Search!

Efisiensi Binary Search

Kekuatan Pembagian Dua

Setiap kali Binary Search melakukan satu perbandingan, ia **mengeliminasi setengah** dari sisa elemen. Inilah yang membuatnya sangat cepat!

Untuk array **1.023 elemen**, hanya dibutuhkan maksimal **10 perbandingan**.

Urutannya: 1023 → 511 → 255 → 127 → 63 → 31 → 15 → 7 → 3 → 1 → 0

Perbandingan Skala Besar

Kompleksitas Binary Search adalah $O(\log n)$ — jauh lebih baik dari $O(n)$!

- Array **1.023** elemen ($2^{10}-1$) → maks **10** perbandingan
- Array **1.048.575** elemen ($2^{20}-1$) → maks **20** perbandingan
- Array **1 miliar+** elemen → maks **30** perbandingan!

Sungguh luar biasa — jutaan data bisa dicari hanya dalam puluhan langkah!

Algoritma Pengurutan (Sorting)

Mengurutkan data adalah salah satu operasi komputasi yang paling sering digunakan dan paling penting dalam dunia nyata.



Perbankan

Bank mengurutkan semua transaksi berdasarkan **nomor rekening** untuk menyiapkan laporan bulanan bagi setiap nasabah.



Perusahaan Telepon

Daftar rekening diurutkan berdasarkan **nama belakang** lalu nama depan, agar mudah menemukan nomor telepon tertentu.



Hampir Semua Organisasi

Hampir setiap sistem — dari e-commerce hingga rumah sakit — perlu mengurutkan data dalam jumlah besar secara rutin.

Selection Sort

Selection Sort adalah algoritma yang mudah dipahami — di setiap iterasi, ia mencari elemen terkecil yang tersisa dan menempatkannya di posisi yang benar.

Cara Kerja

1. Temukan elemen **terkecil** dalam array, tukar dengan elemen pertama
2. Dari sisa elemen, temukan yang terkecil, tukar dengan posisi kedua
3. Ulangi hingga semua elemen terurut

Setelah iterasi ke-*i*, elemen ke-*i* sudah berada di posisi yang benar!

Visualisasi Langkah

34 56 4 10 77 51 93 30 5 52

4 56 34 10 77 51 93 30 5 52 ← tukar 4

4 5 34 10 77 51 93 30 56 52 ← tukar 5

4 5 10 34 77 51 93 30 56 52 ← tukar 10

4 5 10 30 34 51 52 56 77 93 ← selesai!

Implementasi Selection Sort

Kelas `SelectionSortTest` dalam Java mengimplementasikan Selection Sort dengan beberapa metode yang bekerja bersama-sama:

1

`selectionSort()`

Metode **statis utama** yang menjalankan algoritma selection sort pada array `int`. Menggunakan loop bersarang untuk menemukan dan menukar elemen.

2

`swap()`

Metode statis pembantu yang bertugas **menukar nilai** dua elemen array. Menggunakan variabel sementara (`temp`) untuk proses pertukaran.

3

`printPass()`

Metode statis untuk **menampilkan isi array** setelah setiap lintasan, berguna untuk memvisualisasikan proses pengurutan berlangsung.

4

`main()`

Metode utama untuk **menguji** metode `selectionSort` dengan data sampel dan menampilkan hasilnya.

Efisiensi Selection Sort

Kompleksitas: $O(n^2)$

Selection Sort menggunakan dua loop bersarang:

- **Loop luar:** berjalan $n-1$ kali
- **Loop dalam:** berjalan $n-1, n-2, n-3, \dots$ hingga 1 kali

Total iterasi loop dalam: $n(n-1)/2 = (n^2-n)/2$

Dalam notasi Big O, konstanta dan suku kecil diabaikan $\rightarrow O(n^2)$

Artinya untuk Data Besar

Jika kita memiliki 1.000 elemen:

- Sekitar **500.000 perbandingan** diperlukan

Jika datanya 10.000 elemen:

- Sekitar **50.000.000 perbandingan!**

Selection Sort cocok hanya untuk data berukuran **kecil**. Untuk data besar, gunakan algoritma yang lebih efisien seperti Merge Sort.

Insertion Sort

Insertion Sort bekerja seperti cara kita mengurutkan kartu remi — ambil satu kartu, lalu sisipkan ke posisi yang tepat di antara kartu yang sudah terurut.

Cara Kerja

1. Ambil elemen kedua, bandingkan dengan elemen pertama.
Tukar jika perlu.
2. Ambil elemen ketiga, sisipkan ke posisi yang benar di antara dua elemen pertama.
3. Ulangi untuk setiap elemen berikutnya hingga semua terurut.

Pada iterasi ke- i , i **elemen pertama** sudah terurut dengan benar!

Visualisasi Langkah

34 56 4 10 77 51 93 30 5 52 ← awal

34 56 4 10 77 51 93 30 5 52 ← 56 sudah benar

4 34 56 10 77 51 93 30 5 52 ← sisip 4

4 10 34 56 77 51 93 30 5 52 ← sisip 10

📌 Analogi kartu remi: kamu mengambil kartu satu per satu dan langsung menyisipkannya ke tempat yang tepat di tangan kirimu.

Implementasi Insertion Sort

Kelas `InsertionSortTest` dalam Java mengimplementasikan Insertion Sort dengan tiga komponen utama yang bekerja bersama:



`insertionSort()`

Metode statis utama yang mengeksekusi algoritma pengurutan penyisipan pada array bilangan bulat. Mengambil setiap elemen dan menyisipkannya ke posisi yang tepat.



`printPass()`

Metode statis yang menampilkan isi array setelah setiap lintasan pengurutan, membantu memvisualisasikan perubahan posisi elemen secara bertahap.



`main()`

Titik masuk program untuk menguji metode `insertionSort` dengan array data sampel dan menampilkan hasil setiap tahap pengurutan.

Efisiensi Insertion Sort

Kompleksitas: $O(n^2)$

Sama seperti Selection Sort, Insertion Sort juga menggunakan dua loop bersarang:

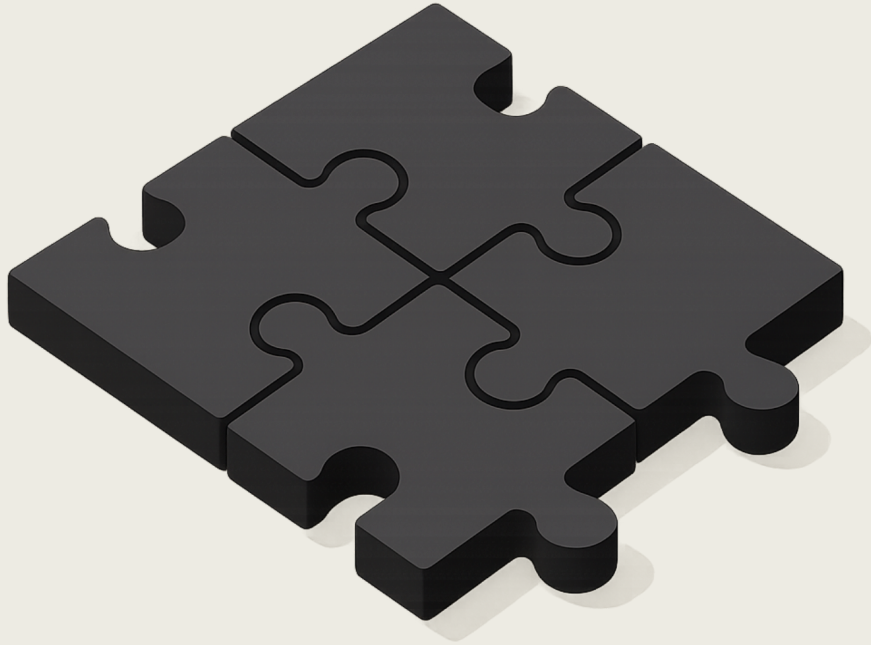
- **Loop for:** mengiterasi sebanyak `data.length - 1` kali (setara $n-1$)
- **Loop while:** mengiterasi elemen-elemen sebelumnya untuk mencari posisi penyisipan

Hasilnya tetap $O(n^2)$ dalam kasus terburuk.

Kelebihan Insertion Sort

Meski sama-sama $O(n^2)$, Insertion Sort memiliki keunggulan praktis:

- Sangat efisien untuk data yang **hampir terurut**
- Lebih cepat dari Selection Sort dalam banyak kasus nyata
- Bisa dijalankan secara **online** (data bisa ditambahkan sambil berjalan)
- Sederhana untuk diimplementasikan dan dipahami



Merge Sort

Merge Sort jauh lebih efisien dari Selection Sort dan Insertion Sort, meskipun cara kerjanya sedikit lebih rumit. Ia menggunakan strategi "**Bagi dan Taklukkan**" (Divide and Conquer).

1

1. Bagi (Divide)

Array dibagi menjadi **dua subarray** berukuran hampir sama. Jika elemen ganjil, satu sisi memiliki satu elemen lebih banyak.

2

2. Urutkan (Conquer)

Setiap subarray diurutkan secara **rekursif** menggunakan Merge Sort yang sama, hingga subarray hanya berisi satu elemen.

3

3. Gabung (Merge)

Dua subarray yang sudah terurut **digabungkan** menjadi satu array terurut yang lebih besar. Proses ini terus berulang ke atas.

Contoh hasil pembagian: 4 10 34 56 77 dan 5 30 51 52 93 → digabung menjadi satu array terurut.

Implementasi Merge Sort

Kelas `MergeSortTest` diimplementasikan dengan beberapa metode yang saling bekerja sama secara rekursif:

01

`mergeSort()`

Metode publik yang menjadi **titik masuk** utama. Memulai proses pengurutan dengan memanggil metode `sortArray`.

02

`sortArray()`

Jantung dari algoritma — menjalankan **rekursi** dengan membagi array menjadi dua bagian dan memanggil dirinya sendiri secara berulang.

03

`merge()`

Menggabungkan dua subarray yang sudah terurut menjadi **satu array terurut**. Ini adalah operasi kritis yang mempengaruhi efisiensi keseluruhan.

04

`subarrayString()`

Menghasilkan representasi **String** dari subarray untuk keperluan tampilan output, memudahkan kita melihat proses rekursi.

05

`main()`

Menguji keseluruhan proses dengan data sampel dan menampilkan hasil setiap langkah.

Efisiensi Merge Sort

Kompleksitas: $O(n \log n)$

Merge Sort jauh lebih efisien! Inilah alasannya:

- Setiap level rekursi membutuhkan total $O(n)$ perbandingan untuk proses merge
- Setiap level membagi array menjadi dua, sehingga ada $\log_2 n$ level
- Total: $n \times \log n = O(n \log n)$

Perbandingan Nyata

Untuk array 1.000.000 elemen:

- Selection/Insertion Sort ($O(n^2)$): ~500 miliar operasi
- Merge Sort ($O(n \log n)$): ~20 juta operasi saja!

Merge Sort kira-kira **25.000× lebih cepat** untuk data besar — perbedaan yang sangat signifikan dalam dunia nyata!

📌 $O(n \log n)$ adalah kompleksitas terbaik yang bisa dicapai oleh algoritma pengurutan berbasis perbandingan — Merge Sort sudah optimal!

Ringkasan: Big O Searching & Sorting

Berikut perbandingan lengkap semua algoritma yang telah kita pelajari, dari yang paling sederhana hingga paling efisien:

Algoritma	Tipe	Big O	Catatan
Linear Search	Pencarian	$O(n)$	Sederhana, tidak perlu data terurut
Binary Search	Pencarian	$O(\log n)$	Sangat cepat, data HARUS terurut
Selection Sort	Pengurutan	$O(n^2)$	Sederhana tapi lambat untuk data besar
Insertion Sort	Pengurutan	$O(n^2)$	Baik untuk data kecil atau hampir terurut
Merge Sort	Pengurutan	$O(n \log n)$	Efisien untuk data besar, lebih kompleks



Massive Parallelism & Parallel Algorithms

Masa depan komputasi bukan hanya tentang algoritma yang lebih pintar — tapi juga tentang **lebih banyak prosesor** yang bekerja bersama-sama secara bersamaan.

Dari Beberapa ke Jutaan Core

Komputer desktop saat ini umumnya memiliki **2, 4, atau 8 inti (core)**. Namun kita sedang menuju era paralelisme masif, di mana satu sistem bisa memiliki **ribuan hingga jutaan prosesor** yang bekerja bersamaan.

Pencarian Paralel Ideal

Dengan paralelisme masif, bayangkan sebuah pencarian di mana **setiap elemen array diperiksa secara bersamaan** oleh prosesor masing-masing. Hasilnya? Pencarian selesai dalam **satu siklus hardware** — kompleksitas $O(1)$ yang sesungguhnya untuk pencarian!

-  Paralelisme masif mengubah paradigma kita tentang efisiensi algoritma — batasan yang berlaku untuk komputasi sekuensial tidak selalu berlaku di dunia paralel.



Terima Kasih!

Semoga materi **Search dan Sort** pada pertemuan ini bermanfaat dan mudah dipahami. Jangan ragu untuk terus berlatih dan bereksperimen dengan implementasi algoritma-algoritma ini!

Search

Linear $O(n)$ vs Binary $O(\log n)$ — pilih yang sesuai dengan kondisi datamu!

Big O

Alat ukur efisiensi algoritma — semakin kecil kompleksitasnya, semakin baik!

Sort

Selection & Insertion $O(n^2)$ untuk data kecil, Merge Sort $O(n \log n)$ untuk data besar.